

Lab05_PedroLacerda

Pedro da Costa Lacerda

```
library(tidyverse)
```

Warning: pacote 'tidyverse' foi compilado no R versão 4.5.3

Warning: pacote 'ggplot2' foi compilado no R versão 4.5.3

Warning: pacote 'tibble' foi compilado no R versão 4.5.3

Warning: pacote 'tidyr' foi compilado no R versão 4.5.3

Warning: pacote 'readr' foi compilado no R versão 4.5.3

Warning: pacote 'purrr' foi compilado no R versão 4.5.3

Warning: pacote 'dplyr' foi compilado no R versão 4.5.3

Warning: pacote 'stringr' foi compilado no R versão 4.5.3

Warning: pacote 'forcats' foi compilado no R versão 4.5.2

Warning: pacote 'lubridate' foi compilado no R versão 4.5.3

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(readr)
library(leaflet)
```

Warning: pacote 'leaflet' foi compilado no R versão 4.5.3

```
library(lubridate)

analisa_aeronave <- function(tail_number, arquivo, arquivo_airports = NULL) {

  # Força o tipo das colunas para evitar conflito entre os pedaços lidos
  esquema_colunas <- readr::cols(
    YEAR = col_double(),
    MONTH = col_double(),
    DAY = col_double(),
    TAIL_NUMBER = col_character(),
    ORIGIN_AIRPORT = col_character(),
    DESTINATION_AIRPORT = col_character(),
    SCHEDULED_DEPARTURE = col_character(),
    DEPARTURE_TIME = col_character(),
    CANCELLED = col_double(),
    DISTANCE = col_double(),
    AIR_TIME = col_double(),
    .default = col_skip()
  )

  # Filtra só os voos válidos do avião em cada bloco lido
  callback_filtrar <- function(dados_chunk, pos) {
    dados_chunk %>%
      dplyr::filter(
        TAIL_NUMBER == tail_number,
```

```

    CANCELLED == 0,
    !is.na(DEPARTURE_TIME)
  )
}

# Leitura em partes para não estourar a RAM
tabela_voos_bruta <- readr::read_csv_chunked(
  file = arquivo,
  callback = readr::DataFrameCallback$new(callback_filtrar),
  col_types = esquema_colunas,
  progress = FALSE
)

if (nrow(tabela_voos_bruta) == 0) {
  warning(paste("Nenhum registro válido encontrado para a aeronave:", tail_number))
  return(list(tabela = NULL, mapa = NULL))
}

# Busca os aeroportos na mesma pasta se o caminho não for passado
if (is.null(arquivo_airports)) {
  arquivo_airports <- file.path(dirname(arquivo), "airports.csv")
}

if (!file.exists(arquivo_airports)) {
  stop(paste("O arquivo 'airports.csv' não foi encontrado no caminho:", arquivo_airports))
}

tabela_aeroportos <- readr::read_csv(
  arquivo_airports,
  col_types = readr::cols(IATA_CODE = col_character(), LATITUDE = col_double(), LONGITUDE = col_double())
) %>%
  dplyr::select(IATA_CODE, LATITUDE, LONGITUDE, AIRPORT, CITY)

# Arruma datas, ordena no tempo e calcula velocidade em km/h
tabela_voos_processada <- tabela_voos_bruta %>%
  dplyr::mutate(
    horario_formatado = stringr::str_pad(as.character(DEPARTURE_TIME), width = 4, pad = "0"),
    data_hora_partida = lubridate::ymd_hm(
      paste(YEAR, MONTH, DAY, horario_formatado, sep = "-"),
      quiet = TRUE
    )
  ) %>%

```

```

dplyr::filter(!is.na(data_hora_partida)) %>%
dplyr::arrange(data_hora_partida) %>%
dplyr::mutate(
  distancia_km = DISTANCE * 1.60934,
  tempo_voo_horas = AIR_TIME / 60,
  velocidade_media_kmh = dplyr::if_else(
    tempo_voo_horas > 0,
    distancia_km / tempo_voo_horas,
    NA_real_
  )
) %>%
dplyr::left_join(tabela_aeroportos, by = c("ORIGIN_AIRPORT" = "IATA_CODE")) %>%
dplyr::rename(lat_origem = LATITUDE, lon_origem = LONGITUDE, nome_origem = AIRPORT, cidade_origem = CITY)
dplyr::left_join(tabela_aeroportos, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) %>%
dplyr::rename(lat_destino = LATITUDE, lon_destino = LONGITUDE, nome_destino = AIRPORT, cidade_destino = CITY)

# Monta o mapa com os pontos e linhas
voos_geolocalizados <- tabela_voos_processada %>%
  dplyr::filter(
    !is.na(lat_origem), !is.na(lon_origem),
    !is.na(lat_destino), !is.na(lon_destino)
  )

mapa <- leaflet::leaflet() %>% leaflet::addTiles()

if (nrow(voos_geolocalizados) > 0) {
  # Desenha os trajetos (linhas mais grossas para voos mais rápidos)
  for (i in seq_len(nrow(voos_geolocalizados))) {
    voo <- voos_geolocalizados[i, ]
    espessura <- if (is.na(voo$velocidade_media_kmh)) 3 else pmax(2, pmin(8, voo$velocidade_media_kmh))

    popup_txt <- paste0(
      "<b>Data/Hora:</b> ", voo$data_hora_partida, "<br>",
      "<b>Origem:</b> ", voo$ORIGIN_AIRPORT, " (", voo$cidade_origem, ")<br>",
      "<b>Destino:</b> ", voo$DESTINATION_AIRPORT, " (", voo$cidade_destino, ")<br>",
      "<b>Velocidade Média:</b> ", round(voo$velocidade_media_kmh, 1), " km/h"
    )

    mapa <- mapa %>%
      leaflet::addPolylines(
        lng = c(voo$lon_origem, voo$lon_destino),
        lat = c(voo$lat_origem, voo$lat_destino),
        weight = espessura,
        popup = popup_txt
      )
  }
}

```

```

        weight = espessura,
        color = "#2B5C8F",
        opacity = 0.5,
        popup = popup_txt
      )
    }

    # Marca os aeroportos visitados
    aeroportos_unicos <- dplyr::bind_rows(
      voos_geolocalizados %>% dplyr::select(IATA = ORIGIN_AIRPORT, lat = lat_origem, lon = lon_origem),
      voos_geolocalizados %>% dplyr::select(IATA = DESTINATION_AIRPORT, lat = lat_destino, lon = lon_destino)
    ) %>% dplyr::distinct(IATA, .keep_all = TRUE)

    mapa <- mapa %>%
      leaflet::addCircleMarkers(
        data = aeroportos_unicos,
        lng = ~lon, lat = ~lat,
        radius = 4, color = "#D95F02",
        popup = ~paste0("<b>", nome, " (", IATA, ")</b><br>", cidade)
      )
    }

    # Retorna a lista nomeada pedida no enunciado
    return(list(tabela = tabela_voos_processada, mapa = mapa))
  }

```

```

# Rodando a função
resultado <- analisa_aeronave(
  tail_number = "N431WN",
  arquivo = file.path("flights.csv.zip"),
  arquivo_airports = file.path("airports.csv")
)

# Conferencia rápida dos resultados
head(resultado$tabela)

```

```

# A tibble: 6 x 24
  YEAR MONTH DAY TAIL_NUMBER ORIGIN_AIRPORT DESTINATION_AIRPORT
  <dbl> <dbl> <dbl> <chr>          <chr>          <chr>
1  2015     1     1 N431WN          DCA            HOU
2  2015     1     1 N431WN          HOU            BOS
3  2015     1     1 N431WN          BOS            BWI

```

```

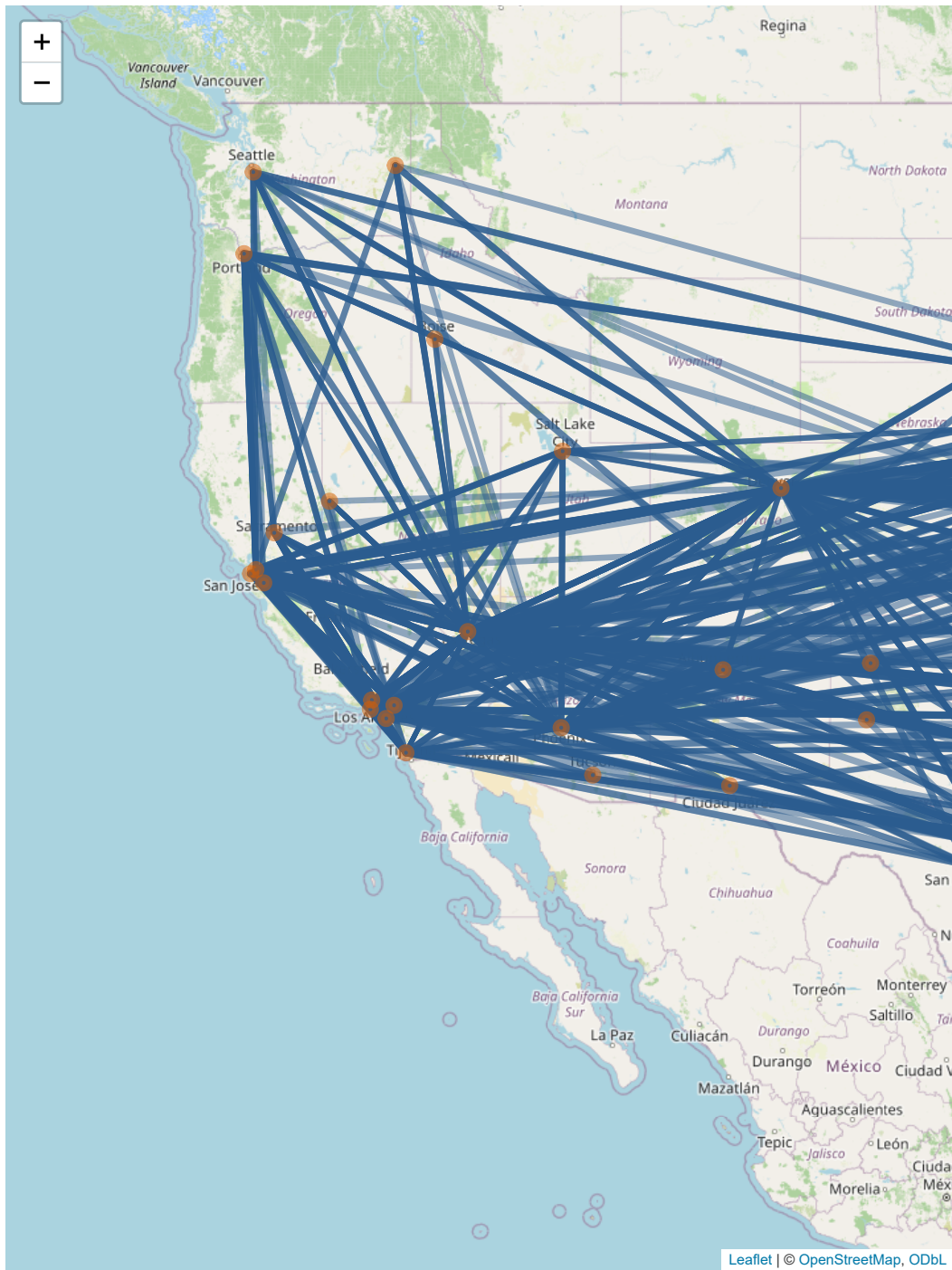
4 2015      1      1 N431WN      BWI          MCI
5 2015      1      2 N431WN      MCI          LAX
6 2015      1      2 N431WN      LAX          LAS
# i 18 more variables: SCHEDULED_DEPARTURE <chr>, DEPARTURE_TIME <chr>,
#   AIR_TIME <dbl>, DISTANCE <dbl>, CANCELLED <dbl>, horario_formatado <chr>,
#   data_hora_partida <dtm>, distancia_km <dbl>, tempo_voo_horas <dbl>,
#   velocidade_media_kmh <dbl>, lat_origem <dbl>, lon_origem <dbl>,
#   nome_origem <chr>, cidade_origem <chr>, lat_destino <dbl>,
#   lon_destino <dbl>, nome_destino <chr>, cidade_destino <chr>

```

```

resultado$mapa

```



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-16 11:59:50 -03"
```